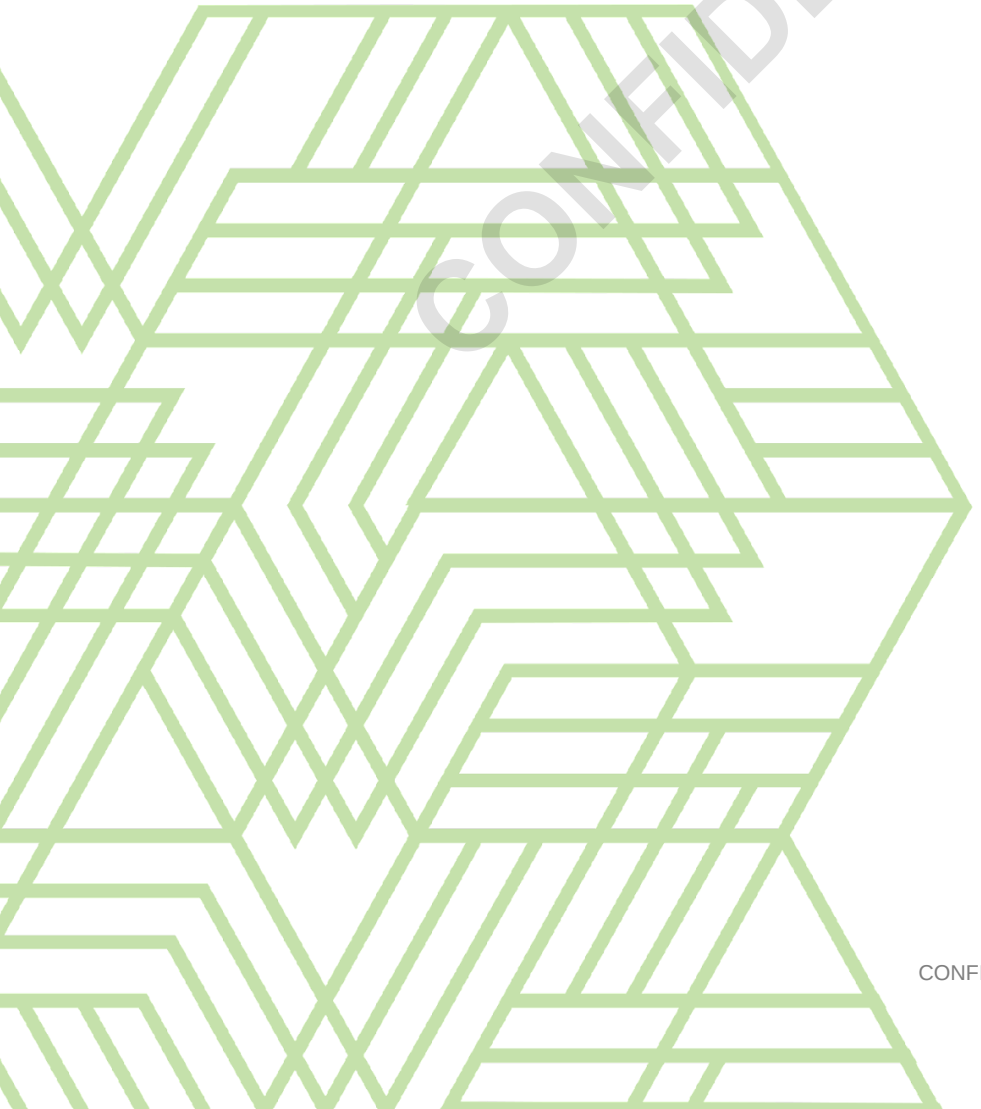
ATM33/e Series Software Development Kit

User Reference Manual

SUMMARY: This document describes the operation of the ATM33/ATM33e Wireless SoC Series. It explains the software environments provided by the Software Development Kit (SDK), the boot processes, and how that enables user-developed code to implement custom Bluetooth LE applications with advanced power management.

CONFIDENTIAL



Atmosic™

Table of Contents

Acronyms and Abbreviations **2**
1. Purpose **3**
2. ROM Boot **4**
2.1 Stack Pointer and Program Counter 4
2.2 Benign Boot Check 6
2.3 Jump to RRAM 6
3. Bootloader **6**
4. SPE and NSPE Images **7**
4.1 MCUboot 8
4.1.1 Image Swap 8
4.1.2 Image Overwrite 9
4.1.3 Secure Boot 10
Signature Verification 10
Signature Data 11
Configuration 12
Signing the Image 12
Adding the Public Key 12
ATMWSTK Verification 13
Anti-Rollback 13
5. RRAM Partitions **14**
5.1 Non-Upgradeable Scheme 14
5.1.1 System Configurations 15
5.1.2 USER DATA 15
5.1.3 NVDS 16
5.1.4 NSPE 16
5.1.5 SPE 17
5.2 Upgradeable Scheme 17
5.2.1 RRAM-Only Scheme 17
5.2.2 RRAM and External Flash Scheme 17
5.2.3 Adjusting the Layout 18
5.2.4 System Configurations 19
5.2.5 ATM Wireless Stack 19
5.2.6 USER DATA 21
5.2.7 MCUBOOT 21
5.2.8 BOOT SCRATCH 21
5.2.9 Active Slot 22
NSPE 22
SPE 22
5.3 OTA Image Upgrade 22
5.4 Secure Journal 23
6. Non-Secure Application **24**
6.1 Build-Time, Read-Only Vectors 24
6.2 HW Interrupt Sources, Routing, and Priorities 24
7. Bluetooth LE Stack **24**

CONFIDENTIAL

8. Power Management **24**
8.1 Bluetooth LE Scheduler 24
8.2 Platform Scheduler 25
8.3 Poke the Watchdog 25
8.4 Power Management 25
8.5 PARAM_ID_SLEEP_ENABLE 26
8.6 Platform Sleep Duration 29
8.7 Backplane Frequency Scaling 29
9. Crypto Accelerators **29**
9.1 AES Engine 29
9.2 SHA2/HMAC 30
10. Secure Debug **30**
Reference Documents **31**
Revision History **31**

CONFIDENTIAL

List of Figures

Figure 1 - SPE, NSPE, and Boot Chain Block Diagram

Figure 2 - Extended Boot Chain Block Diagram

Figure 3 - ECDSA Signature Generation Block Diagram

Figure 4 - ECDSA Verification Flow Diagram

Figure 5 - Basic Non-upgradeable Partition Scheme Block Diagram

Figure 6 - Up-gradeable Partition Scheme Block Diagram

Acronyms and Abbreviations

Acronyms	Definition
AES	Advanced Encryption Standard
ATMWSTK	ATM Wireless Stack
CBC	AES Cipher Block Chaining
CFB	AES Cipher FeedBack
CTR	AES Counter Mode
DAP	Debug Access Port

Acronyms	Definition
ECB	AES Electronic Codebook
ECC	Elliptic Curve Cryptography
ECDSA	Elliptical Curve Digital Signature Algorithm
FIPS	Federal Information Processing Standards
HMAC	Hash-based Message Authentication Codes. HMAC-256 is based on SHA2 (SHA256).
NIST	National Institute of Standards and Technology
NSC API	Non-Secure-Callable Application Interface
NSPE	Non-Secure Processing Environment
NVDS	Non-Volatile Data Storage
OFB	AES Output Feedback Mode
OTA	Over-The-Air
OTP	One Time Programmable
PC	Program Counter
ROM	Read-Only Memory
RRAM	Resistive Random Access Memory
SHA2	Secure Hash Algorithm 2
SP	Stack Pointer
SPE	Secure Processing Environment
WFI	Wait for Interrupt
VSTOR	Vector Table Offset

CONFIDENTIAL

1. Purpose

This document describes the operation of the ATM33/e Series and the software environments provided by this SDK, their boot processes, and how that enables user-developed code to implement custom Bluetooth LE applications with advanced power management.

Applicable Devices:

Device	SoC	Package
ATM33e Series	ATM3330E-5DCAQN	ATM3330e 7x7 mm QFN package
ATM33 Series	ATM3330-5DCAQN	ATM3330 7x7 mm QFN package
	ATM3325-5DCAQK	ATM3325 5x5 mm QFN package
	ATMEVK-3325-LQK	ATM3325 5x5 mm QFN with extended storage package

CONFIDENTIAL

2. ROM Boot

This section describes the boot process of the ARM Cortex-M33F CPU. This boot can happen for a number of reasons including: power-on, external pin reset, software reset, watchdog reset, leaving hibernation, etc.

2.1 Stack Pointer and Program Counter

Immediately after reset, the ARM Cortex-M33F vector table maps directly to the beginning of the ROM device. First, the Stack Pointer (SP) is loaded from address 0x10000000. For this version, that ROM value is 0x30004000. The CPU also loads the Program Counter (PC) from the reset vector at address 0x10000004. That ROM value is 0x10000741, which is the beginning of the reset handler, also located in the ROM device.

For more information, see the `Vector tables` section of the Armv8-M Architecture Reference Manual.

Refer to the Reference Documents section for more information.

2.2 Benign Boot Check

The first few instructions of the ROM's reset handler provide a mechanism to guarantee that the Debug Access Port (DAP) can gain control of the ARM Cortex-M33F after a reset. When P25 is asserted, this code will execute a Wait for Interrupt (WFI) instruction. This will idle the CPU and, until it is reset, keep it off all buses.

The Evaluation Board (EVB) design includes a weak pull-down on P25 to allow the CPU to continue the boot process.

Correspondingly, during a soft reset (that only resets the CPU but not the peripheral modules), the ROM boot will impact the following peripheral registers:

- CMSDK_WRPR0 PSEL_G set to 0
- CMSDK_GPIO1 input enable cleared for 0x0200 (P 25)

2.3 Jump to RRAM

Next, the Vector Table Offset Register (VTOR) is pointed to the beginning of RRAM (address 0x10010000), the SP is loaded from the first word of the VTOR and the PC is loaded from the second word. This simulates booting from a standard ROM image, but one that is located in RRAM.

The rest of the boot process is determined by the specific software image that was previously programmed into RRAM.

CONFIDENTIAL

3. Bootloader

The bootloader application is provided to ARM TrustZone-M capable platforms as an extension to the ROM boot process. The boot ROM executes a secure application found at the beginning of RRAM. A secure application however is expected to bootstrap a non-secure application at some point. The bootloader is provided as an example of that process. The bootloader is intended to complete system initialization, partition resources for the nonsecure application, and jump to it. Once inside the application, the processor has fully switched to nonsecure mode. It can return to secure mode (via handlers in the bootloader) through secure exceptions, interrupts, and non-secure callable gateway entry points (explained further below). For more information please refer to *Arm® TrustZone Technology for the Armv8-M Architecture* in the Reference Documents section.

4. SPE and NSPE Images

The examples in the SDK are configured to be built as an application running in a Non-Secure Processing Environment (NSPE). NSPE and NSPE image (an image compiled to run in an NSPE) will be used interchangeably. The bootloader executes in a Secure Processing Environment (SPE) and can be referred to as the SPE image or simply the SPE. In the Atmosic platform, there can only be one SPE and one NSPE. The SPE differs from a traditional bootloader in one main regard. Typically, bootloaders transfer full control to the application (or kernel) and no vestige remains. An SPE however is part of the application that handles secure exceptions and can provide secure application services to the NSPE.

The SPE can operate secure devices, handle secure interrupts, as well as implement exception handlers for various system and security faults. The SPE in its current form provides basic I/O and memory partitioning and default handlers for secure exceptions. These handlers provide default handling and traceback for exceptions generated by the NSPE image if it attempts to access a resource outside its security boundary. The SPE is designed to be lightweight with a minimal C run-time and a minimal set of secure services mainly around cache management. In the future, the SPE can be configured with additional secure services to meet security requirements. The SPE offers developers the ability to isolate critical security functions in a separate execution domain.

CONFIDENTIAL

The SPE provides secure functions that are callable from non-secure applications to access secured resources. These non-secure-callable APIs (NSC APIs) provide services such as secure peripheral I/O requests or cache maintenance where control can only be performed while in secure mode. The SPE uses a fixed size reservation to hold both main SPE code and a special non-secure callable region at the end of the SPE image. The NSC region is a specially marked region that contains the secure gateway instructions that allows the NSPE to transition into secure mode and call a secure function. In the bootloader code you can refer to these examples that provide cache maintenance functions (see /examples/bootloader/bootloader.c) :

```
secure_icache_flush
secure_icache_disable
```

The following diagram shows how the SPE and NSPE are stacked and the boot chain starting at the ROM.

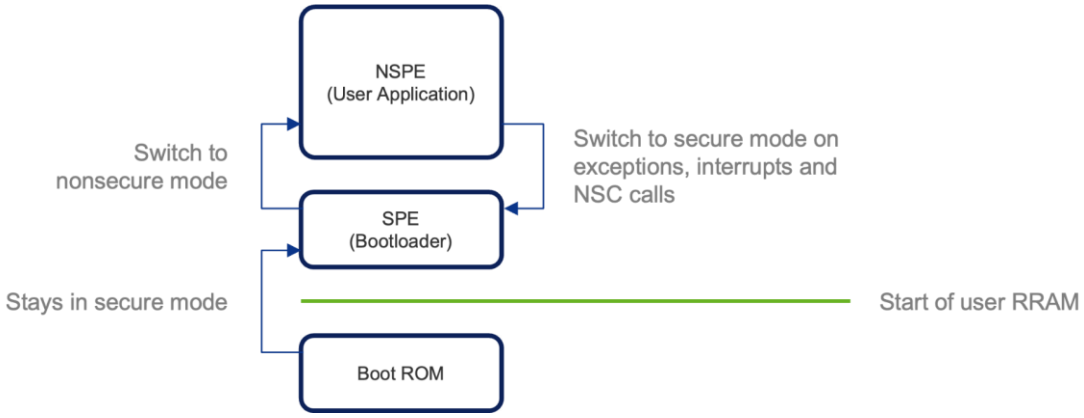


Figure 1 - SPE, NSPE, and Boot Chain Block Diagram

The processor operating in the NSPE image can transition back to the SPE image through exceptions and NSC calls. It cannot, however, transition back into the boot ROM. The boot ROM does not implement NSC callable regions.

4.1 MCUboot

The SDK provides an implementation of the open source MCUboot bootloader (see https://www.mcuboot.com) with reliable image swapping and SHA2 based integrity checks. This bootloader is used when an upgradeable partition scheme is required to support over the air (OTA) image updates. MCUboot is not intended to be customized for each application. It provides boot (secure or nonsecure) and image swapping and image overwrite services. Users should consider MCUboot an extension of the ROM and could be ROMed in future revisions of the SoC. Extensions of the boot process (boot chaining) can be done with the bootloader or SPE described in the previous section. The extended boot chain is shown in Figure 2.

CONFIDENTIAL

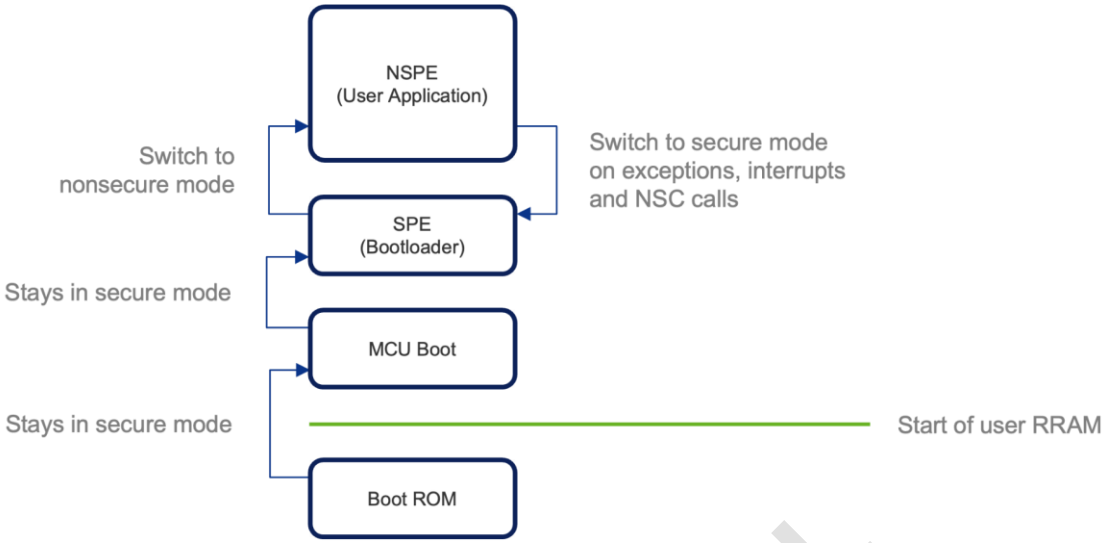


Figure 2 - Extended Boot Chain Block Diagram

CONFIDENTIAL

The next two sections describe the two available upgrade strategies: swap and overwrite. Both strategies apply to RRAM-only and RRAM+external flash based configurations.

4.1.1 Image Swap

Image swapping involves transferring an image byte-for-byte between an active slot and an inactive staging slot. Swapping allows the existing image to remain in the system (swapped to the inactive slot) in the event it needs to be recovered. This recovery mechanism is covered in the OTA Users Guide. The image transfer is done reliably through a partition called the BOOT SCRATCH. The BOOT SCRATCH journals the transfer and allows the swap to be fault tolerant on power loss or system crashes. Images can be restored (swapped back) in the event of a boot failure or failure to acknowledge an image is good during the OTA process.

4.1.2 Image Overwrite

The overwrite strategy simply applies the upgrade without using the swap mechanism. This can shorten installation time at the risk of installing a non-functioning image. Care should be taken when this strategy is used as the device has no recourse if the image does not boot. The image overwrite strategy does not employ a scratch buffer and this resource can be recovered for application use.

Both swap and overwrite strategies are compatible with secure boot and utilize the same integrity checking mechanisms. All images found in the upgrade staging areas are verified (hashed and optionally signature checked) before being applied.

4.1.3 Secure Boot

The secure boot process starts with the ROM boot. The ATM33/e ROM boot process is minimally designed to read a vector table at a fixed location at the beginning of RRAM. This RRAM contains the first stage bootloader in the boot chain. Due to the complexities of tailoring a ROM-based secure boot for each customer application, RRAM was selected to extend the functionality of the ROM. The location of the RRAM bootloader can be locked down as read-only memory using OTP within the first 64 KB of RRAM.

The SDK contains a secure boot loader firmware reference based on MCUboot (see https://docs.mcuboot.com/design.html). MCUboot also provides upgrade services to swap and test an OTA upgrade that has been installed by the main application. Secure boot verifies the integrity of the upgraded image before the image swap operation and for all subsequent system boot checks.

CONFIDENTIAL

Signature Verification

The default signature verification algorithm uses elliptic curve cryptography (ECC) digital signature verification algorithm (ECDSA). ECC has the benefit of smaller public keys compared to other cryptographic methods such as RSA. ECC is widely used in resource constrained embedded devices for this reason. The selected curve domain parameters used are based on NIST (FIPS) P-256. The curve is chosen due to its standardization and wide availability of implementation in security tools, OSes, and security products. It is the underlying curve used in Bluetooth LE secure connections. Only the ECDSA-verify portion is required and implemented in MCUboot. Currently, the ECDSA algorithm is performed in software using open source libraries provided by MCUboot.

Signature Data

The signature data components are generated by the trusted signing servers. For development purposes the SDK provides development keys when signing images in a development environment. Please make note of the warning message when signing images. The development keys should never be used in production and the signing process should be done in a secured environment. The signature uses a random

number, the selected elliptic curve domain parameters, a private key, and the image hash (SHA256) to generate an ECDSA signature as shown in Figure 3:

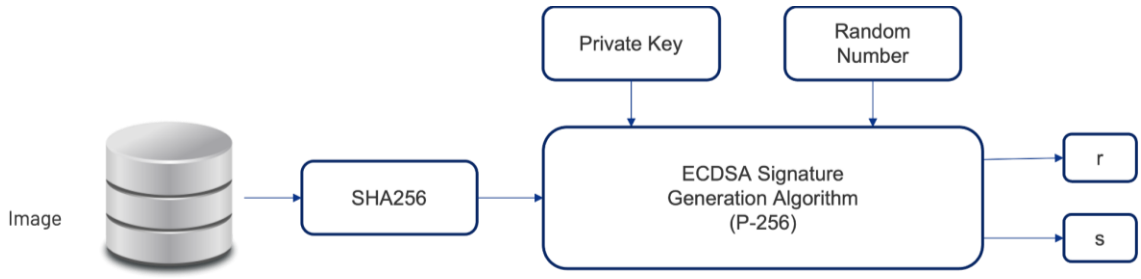


Figure 3 - ECDSA Signature Generation Block Diagram

ECDSA signatures consist of two integers r and s and are referred to as the `signature pair`. The value of `r` is derived from a random number and the elliptic curve domain parameters (i.e. NIST P-256). The value of `s` is derived from `r`, the private key, and the image hash. The integers r and s are included in the signature block. This signature block is written to meta data contained within the image binary. The image binary can be loaded onto the device and MCUboot will verify the signature at boot. For a signed image binary delivered via OTA, the image signature is checked in MCUboot's image upgrade process and before the image update is applied.

MCUboot will perform the signature verification based on the public key that is currently built into the image. In future versions of the SDK, this key can be programmed into the Secure Journal and locked down. The signing pair (r,s) is read from the image meta data and the SHA256 is computed just in time for the image. See Figure 4.

CONFIDENTIAL

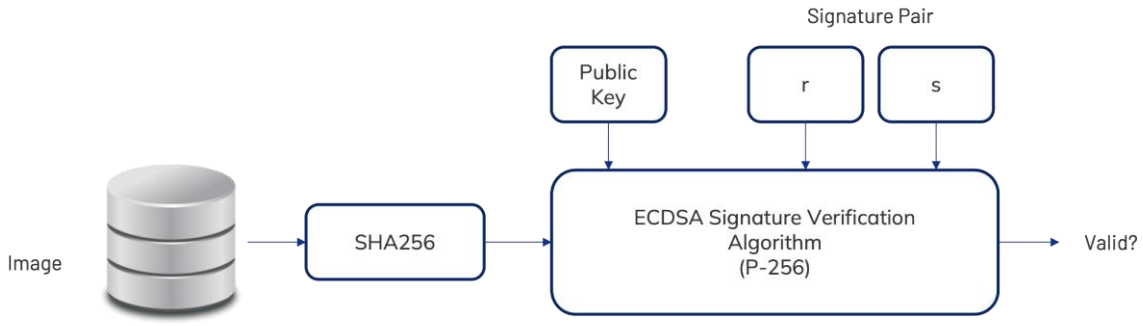


Figure 4 - ECDSA Verification Flow Diagram

ATM33/e provides SHA256 hardware accelerators to make this process fast and efficient. The output of the verification is a simple yes/no verification that the signature

pair is cryptographically associated with the just in time computed hash. If the signature check fails, MCUboot will halt and wait for a system reset through the boot watchdog.

Configuration

Secure boot requires the use of MCUboot when building and installing an application.

All examples can be built with **secure boot** enabled by adding the following build time options:

```
USE_MCUBOOT=1 USE_SECURE_BOOT=1 <SECURE_BOOT_KEY='pem file' >
```

- `USE_MCUBOOT=1` is used to enable MCUboot in our boot chain.
- `USE_SECURE_BOOT=1` is used to enable **secure boot**
- `SECURE_BOOT_KEY=<pem file>` is an optional variable to choose a specific .pem file. A .pem file is a public/private key certificate file generated by tools such as openssl. The default uses the MCUboot development public key certificate which is widely distributed.

Do not use the default PEM file in production. This is only for development purposes and widely distributed in SDK releases.

CONFIDENTIAL

Signing the Image

When signing the image, the process is similar to a non-secure MCUboot build, however, the signing tool will append additional ECDSA signature information in the image metadata.

Adding the Public Key

The private key is used to sign the image, however, to actually verify the image, MCUboot needs access to the public key used for ECDSA verification. MCUboot by default supports compiling the public key into the image itself. As part of the build process, a `public_keys.gen.c` file is generated by the imgtool and it is compiled into MCUboot. From there, MCUboot can use the key to validate the integrity of the firmware populated into the active image slot as well as upgrades ready to be swapped from the staging slot (See section Upgradeable Scheme).

ATMWSTK Verification

When the ATM Wireless Stack (ATMWSTK) is used and populated in the stack partition area, the integrity of this stack is checked by the SPE (bootloader) image. At build time the SPE and the NSPE are built against a specific version of the wireless stack provided in the SDK. The SDK provides the known hash value of the selected stack variant and this value is compiled into the SPE. The SPE will use the SHA256 accelerator to confirm that the wireless stack SHA calculation matches the expected value. Since the SPE is part of the image that is signed, the SHA value used to verify the stack is also trusted and authenticates the wireless stack before the NSPE image is started.

Anti-Rollback

Anti-Rollback protection is a security measure that is used to harden against attacks that attempt to revert to older (less secure) firmware. Secure boot provides guarantees against untrusted software from being programmed onto the device, however, it cannot protect against older trusted firmware with a known security flaw. Anti-rollback protects against this by signing each image with a security version counter that is independent from the product-specific firmware versioning metadata. MCUboot prevents 'downgrades' to an image with a lower security version. However, it does allow 'downgrades' to an older image with the same security version.

Anti-rollback is enforced using a hardware based security counter which provides a monotonic, persistent counter to compare against the signed security version. If an image with a **HIGHER** security value is uploaded, verified, swapped, and marked as a valid image, the local secure counter will be incremented to match this new value. This will set the minimum security level for all future upgrades.

When building your application, add the following command line options to enable anti-rollback:

```
USE_ROLLBACK_PROT=1 SECURITY_RB_VERSION=<value>
```

The value of SECURITY_RB_VERSION is a positive value from 0-2047. Secure counters saturate at the value of 2047. This value has no special meaning other than higher values indicate a higher level of security. If this value is set to '1', then all future upgrades must be '1' or higher after this image has been successfully installed on the device.

When using this feature, be aware that when the secure counter fuse in OTP is blown, the counters are protected and operate monotonically. Using anti-rollback on devices that have this OTP configuration is permanent and irreversible.

CONFIDENTIAL

5. RRAM Partitions

The on-chip RRAM resources are divided into partitions to store code and data for the SPE and NSPE applications. This comprises the working application. There are two main partitioning schemes supported in the current SDK: non-upgradeable and upgradeable. When the upgradeable partition scheme is employed, an additional bootloader (MCUboot) and wireless stack partitions are included.

5.1 Non-Upgradeable Scheme

A non-upgradeable partition scheme supports applications that are not field-upgraded during the product's life cycle. This disables the ability to provide upgrades over wireless (OTA) methods. This does not, however, exclude the use of the debugger interface (via cable and debug pins) to field upgrade units. The non-upgradeable partition scheme provides the most RRAM resources for your application. This includes enhanced debug and access to all features of the wireless stack. During initial development, a non-upgrade scheme provides the most flexibility and debug options as you work towards your final application. The SDK allows you to switch to the upgradeable scheme, with some restrictions, at some point in your development cycle. Please refer to the upgradeable scheme section for details.

Figure 5 provides an overview of a basic non-upgradeable partition scheme.

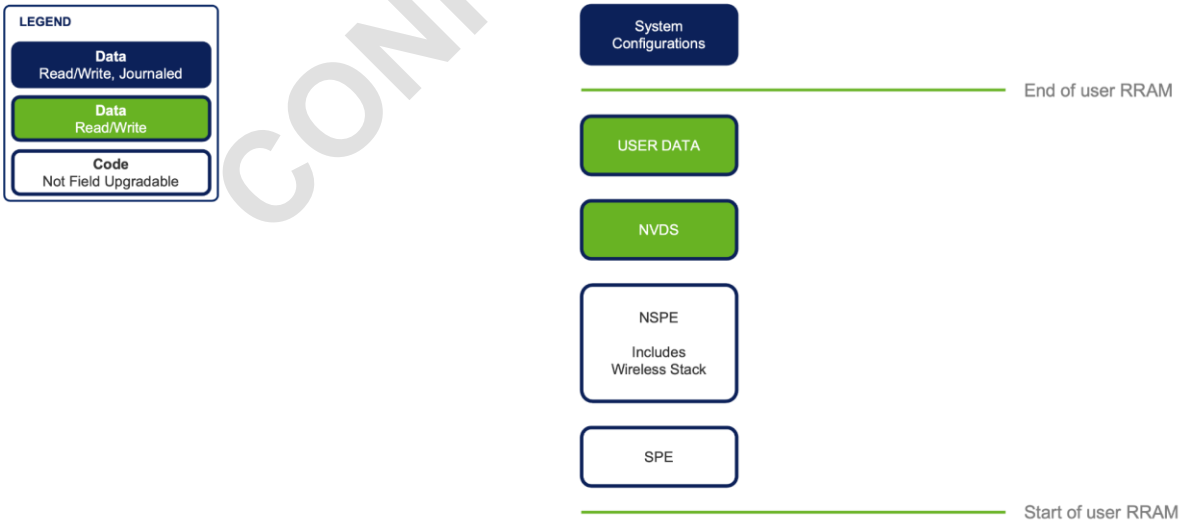


Figure 5 - Basic Non-upgradeable Partition Scheme Block Diagram

Each partition is described in the following sections.

CONFIDENTIAL

5.1.1 System Configurations

The system configuration partition is at a fixed location at the top of the RRAM memory map. Its size is specified by the chip version and typically 2 KB. This area may contain chip data and system configurations that are critical to the secure and reliable operation of the device. This may include but not limited to:

- Per-chip ATE Data
- XTAL configuration/calibrations
- RF calibrations
- Optional Public keys and secure boot options
- Unique identifiers (board addresses, serial numbers)
- Limited USER DATA

This block of memory can be made immutable and secure through manufacturing programming. It operates similarly to one-time-programmable (OTP) memory using a write-protected journaling mechanism (see Secure Journal).

CONFIDENTIAL

5.1.2 USER DATA

The USER DATA is an optional partition to reserve an area of RRAM to be managed by the application. This area is not programmed by any SDK tools and is only held in reserve. It is assumed that any provisioning of this area is handled by the customer in their manufacturing process. Applications can obtain the location and size of this area at compile time. The size of this partition is configured with the USER_SIZE setting. The location of this area is determined by the size of the SPE, NSPE, and NVDS reservations with the remaining balance dedicated to the USER DATA area.

5.1.3 NVDS

This is the Non-Volatile Data Store that stores application settings that can be altered in the field via application software. NVDS stores parameters such as Bluetooth pair and bonding settings obtained during normal operation. NVDS parameters can be added, removed, or modified through the NVDS APIs under the direction of the application or other system components. The size of this partition is configured with the NVDS_SIZE

setting. The location of NVDS is determined by the amount reserved for the NSPE and SPE. At this time NVDS can only be located in RRAM.

5.1.4 NSPE

Please refer to an earlier section on the Bootloader for additional information. The NSPE hosts its own application operating within a security boundary enforced by hardware. The NSPE image is where most of the user application, drivers, libraries, and wireless stack code reside. In the future, some drivers and libraries may operate exclusively in the SPE application to provide secure services through a secure gateway. Most users will be developing their applications as NSPE applications. In more advanced security applications additional development can be done to expand the SPE's scope. The SDK provides a basic SPE that configures most I/O and memory resources for NSPE use. This eliminates additional user configurations and helps transition users to the concept of an NSPE image.

5.1.5 SPE

The SPE image contains additional system startup code along with a handful of secure services. The default SPE provided in the SDK is the bootloader that boots the NSPE image.

5.2 Upgradeable Scheme

If the product requires field upgrade capabilities, it can be switched to an upgradeable scheme with the following build options: `USE_MCUBOOT=1 <other options>`. This will configure and use MCUboot with an upgradeable partition scheme. See Figure 6. MCUboot requires the installation of additional Python dependencies. Please refer to the tools README on these dependencies.

Upgrades schemes supported are RRAM-only or RRAM supplemented by external SPI flash.

5.2.1 RRAM-Only Scheme

This scheme utilizes the internal RRAM memory to host both the application image and any storage to hold an upgrade image. Take care that your application will need to fit within the constraints of this scheme as additional RRAM resources are consumed to provide reliable and reversible image upgrades. The key restrictions are:

CONFIDENTIAL

- Reduced available RRAM for the application code.
- Reduced debuggability of the application code due to limited console log use.
- A switch to the ATM Wireless Stack configured for a reduced wireless feature set and debug support.
- Hard partitioning is incurred when the design is finalized and ready for production.

It is highly recommended to begin the development in a non-upgrade scheme while being aware of the limitations of the upgradeable scheme that utilizes RRAM only.

5.2.2 RRAM and External Flash Scheme

To expand the capacity of RRAM in order to host larger images, external flash can be added to the system and operate as a data store for the OTA staging area and BOOT SCRATCH area. The application still executes from RRAM and external flash is only used for OTA data storage. The key restrictions and features are:

- External flash is attached to the QSPI port. Other SPI ports are not supported at this time.
- External flash is only used to store upgrade images that are to be installed.
- External flash does not support execute-in-place code and can only be used as a data store.
- Hard partitioning is incurred when the design is finalized and ready for production.
- Application images can be built with the full capabilities of the BLE stack without compromise of feature sets. The use of a feature-reduced ATM wireless stack is not required.
- Full upgrade of both the application and BLE stack in a single monolithic image.
- Due to the increased image size, care should be taken as the upgrade image will now take longer to transfer and longer to install with the available upgrade strategies.

An upgradeable partitioning scheme is shown below:

CONFIDENTIAL

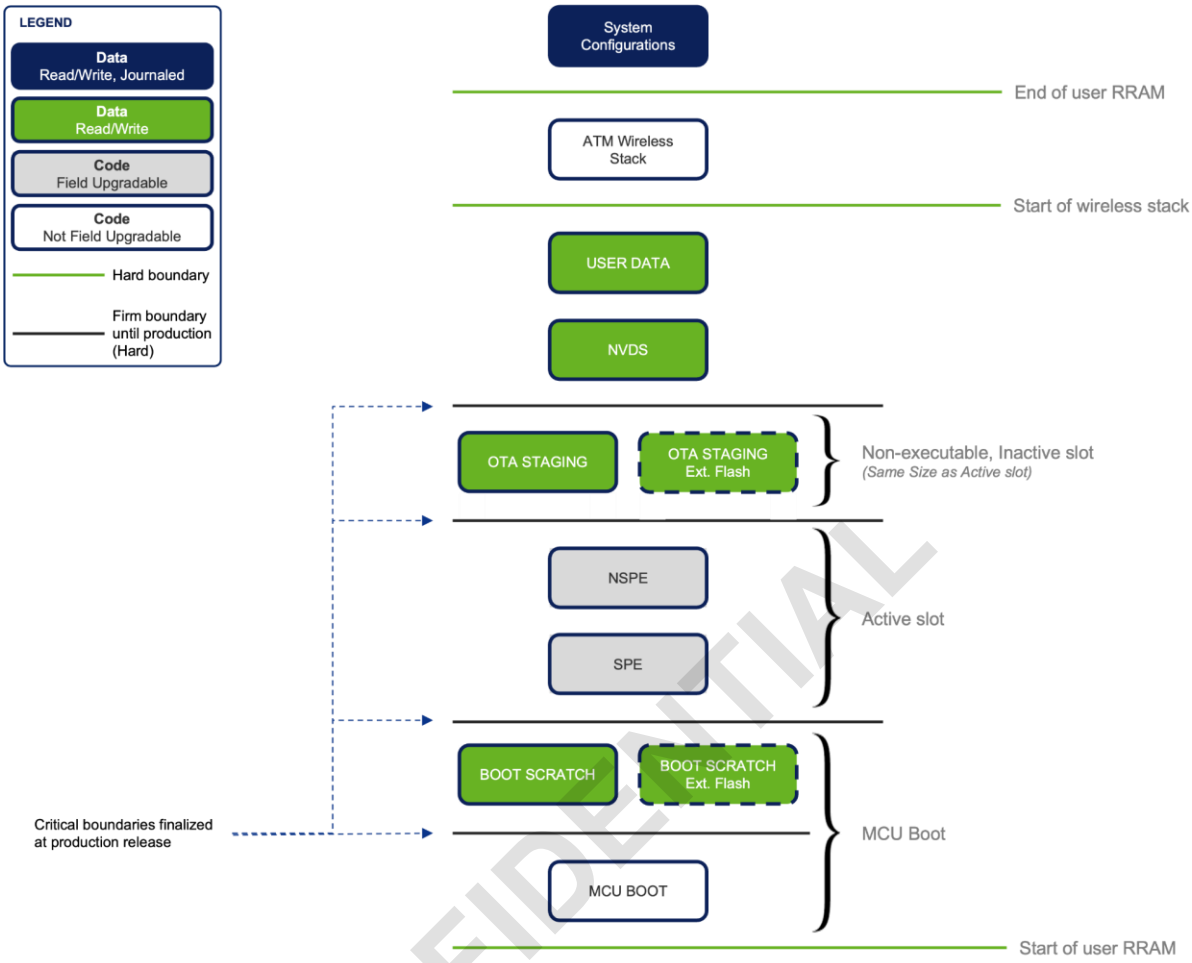


Figure 6 - Upgradeable Partition Scheme Block Diagram

CONFIDENTIAL

5.2.3 Adjusting the Layout

The user can configure the sizes of the following partitions to establish a baseline layout conducive to image upgrades:

- NVDS
- USER DATA
- BOOT SCRATCH (only if image swap is used)

The user will have limited control over the size of the following components (based on required features):

- ATM Wireless Stack (fixed binary image at a specific location, see Figure 6)
- MCUBOOT (minimum binary image to support image swapping, see Figure 6)

The combination of the user configurable partitions and the minimum working set for the wireless stack and MCU bootloader will determine the final partition layout. During development the blue lines can move (and compress regions) as you develop your application and optimize the layout. However, once your product goes to production, the '**firm'** boundaries become fixed ('**hard'**) for the life of the product. It is imperative that you adjust partitions such that data or code can safely expand into unused space within their assigned partition. Once fixed, they cannot be altered in the field. The adjustability and tradeoffs for each of these areas are described in each of their respective sections below.

5.2.4 System Configurations

This partition is identical to the non-upgradeable scheme. This partition cannot be adjusted.

5.2.5 ATM Wireless Stack

This is a distinct partition to host the wireless stack (i.e. Bluetooth Low Energy). This partition is mainly used in RRAM-only schemes where RRAM resources are limited. It is not required for RRAM+Flash schemes as additional RRAM resources are available to support a monolithic application that includes both application and stack. The ATM wireless stack is populated at a fixed location and is sized for a reduced feature set and reduced debug capabilities. The stacks are built as release images only and debug build versions are not provided. It is hosted at the end of memory adjacent to the system configuration area and is not relocatable. The SDK may contain multiple variants of the wireless stack where each variant provides additional features at the expense of RRAM usage. During initial development the wireless stack may grow downwards with new SDK releases (bug fixes and enhancements to the stack). The NSPE image (user application) employs **static** linkage to the wireless stack binary. This is an important detail that affects OTA updates. An updated image must be built with the same SDK release that sourced the ATM Wireless Stack binary that was pre-installed on the device. A stack check (SHA256 hash) may fail if the upgrade image expects a different (i.e. newer) version of the stack.

In secure boot operation, the stack will be authenticated/verified along with the SPE and NSPE images. Once the stack is selected for production deployment, its location and composition is forever locked in place for the life of the product. **ATM Wireless Stack is not field upgradeable after production.** The OTA staging area only has enough room to stage the user's SPE/NSPE images. For wireless stacks that have already been

CONFIDENTIAL

programmed in production and now distributed into the field, updates and bug fixes to the stack will rely on patching libraries that are built into the NSPE application and applied when the NSPE image is upgraded. These patches affect the run-time operation of the stack without altering the ATMWSTK stack binary. This is similar to ROM patching used in earlier ROM-based products (ATM2).

In a RRAM+External Flash scheme the application links in the BLE stack and forms a monolithic image. In this scheme the BLE stack is fully upgradeable with the application.

5.2.6 USER DATA

Refer to the earlier description. The size of the USER DATA can affect the amount of space available to other partitions. USER DATA is not modified or mirrored by upgrades. Consider this a reserved area only for application use. **The USER DATA location and size is not adjustable after production. At this time USER DATA can only be placed in RRAM.**

5.2.7 MCUBOOT

This partition holds the MCUboot image. **This image is not field upgradeable.**

5.2.8 BOOT SCRATCH

The size of the scratch impacts the number of upgrades that are possible given the maximum write endurance and the average size of the upgrade images. For RRAM only schemes the boot scratch resides in RRAM. All upgrade image data passes through the scratch area during the swap operation. RRAM has an endurance of ~10000 write cycles. A 16 KB scratch would allow up to 625 upgrades when the average size of the upgrade is 256 KB (16*10000 / 256). A recovered (swapped back) image counts as an upgrade. The image size to scratch size ratio is the number of times the scratch will be overwritten on every upgrade. Thus a scratch size of 8 KB would allow for ~312 upgrades and 4 KB, ~156 upgrades, assuming an upgrade size of 256 KB. The scratch size requires a minimum of 2 KB and can be increased in 2 KB increments. **For an RRAM+Flash scheme the boot scratch resides in flash memory and the write endurance for the scratch improves to 100,000 cycles for typical flash devices. The minimum scratch size in flash is 4KB and can be increased in 4KB increments.**

CONFIDENTIAL

Once these partitions are configured for production use, they are forever fixed for the life of the product. **The location and size of the BOOT SCRATCH (if used) is also fixed in place.**

To maximize the available space for a swappable application image, the user should consider the following:

- Minimizing the use of NVDS data.
- Minimizing the use of USER DATA (can be eliminated if not required)
- Sizing the BOOT SCRATCH (if used) for the image size and expected number of upgrades for the life of the device.
- For RRAM-only schemes, selecting the ATM Wireless Stack that meets minimum requirements for the application.
- For RRAM+Flash schemes, selecting a flash device that matches the size of the image that can be upgraded in RRAM.
- Selecting MCUboot features that meets a minimum level of security requirements.

The next section describes the RRAM partitions within the active and inactive slots. In MCUboot terms, the active slot is synonymous with the `primary slot` with the inactive slot the `secondary slot`. Images only execute when they are in the primary slot (i.e. they are active). An OTA update is delivered to the inactive slot whereby on the next boot to MCUboot, the data therein is swapped or overwritten to the active slot. The inactive slot is a read/writeable data region that stages the upgrade image but also stages the recovery image when image swapping is enabled. The active slot size must equal the inactive slot size.

The next section describes how each slot is subdivided.

5.2.9 Active Slot

NSPE

This is the same NSPE in the non-upgradeable scheme. Please refer to the section below on how this partition is delivered via OTA.

SPE

This is the same SPE in the non-upgradeable scheme. Please refer to the section below on how this partition is delivered via OTA.

CONFIDENTIAL

5.3 OTA Image Upgrade

An OTA upgrade is delivered as a binary image containing both the SPE/NSPE images targeting the code regions in the active slot. The images are hashed/signed together into a single binary that can be overlaid into the inactive slot and swapped or overwritten into the active slot. This combined binary is located in the application (examples) folder as signed_<example name>.bin. This combined signed image is also used by debug tools to install the image on target into the active slot (the initial installation of the image, pre-OTA). After making software changes and rebuilding, the regenerated binary can be used by an OTA client-side application (mobile application or SDK OTA demo tools) to install a new image into the inactive slot for swapping. The image does not contain an NVDS database. The NVDS and USER DATA locations are fixed after production and cannot be resized using OTA.

When an OTA image is swapped, the NVDS partition in the active slot is left intact and not swapped. The new swapped image will see the same NVDS. NVDS mirroring has been deprecated for ATM33/e products. Customers should use the USER DATA partition for custom data items. Additional details can be found in the ATM33/e Series OTA Update Service User Guide available on the Atmosic Support website.

5.4 Secure Journal

Secure Journal is a dedicated region of RRAM that stores system parameters that are accessible only in secure mode. The region is located at the secure address of 0x1008_F800 and is 1776 bytes long. A secure service is provided in the SPE (bootloader) that can access the contents of the journal. A non-secure journal driver is provided for software to retrieve data from the journal in a secure manner.

The Secure Journal uses a basic TLV (type-length-value) encoding scheme. The data stored in the Secure Journal is intended to be prewritten and only read on chip. To enforce this, the secure data space (upper 2 KB of RRAM) is designated as a secure region via the Trustzone Security Attribution Unit (SAU). This prevents non-secure applications from writing arbitrary data into the journal and incrementing the secure ratchet. Non-secure callable functions are provided by the SPE to access the Secure Journal and read out NVDS tags. For more information on non-secure callable functions, please see SPE and NSPE Images section. Currently, the Secure Journal only supports device parameters such as CAL, board address, and ATE data.

CONFIDENTIAL

There is no support for programming the Secure Journal using an embedded application. The journal is mainly intended for system configurations programmed separately from the running embedded application.

6. Non-Secure Application

`startup_ARMv8MBL.c` sets up the machine state for a C-language program. There are two main parts:

1) Exception vector table, including initial stack pointer and reset handler
2) C library init, which includes such things as data copying, bss clearing, constructor invocation, and calling main()

6.1 Build-Time, Read-Only Vectors

The secure and non-secure application startup code provided by this SDK utilizes the linker to produce a vector table in RRAM, which is non-volatile and write protected. The VTOR register stays pointed to this read-only table, so CMSIS functions such as NVIC_SetVector() will have no effect.

6.2 HW Interrupt Sources, Routing, and Priorities

All peripheral interrupt sources are home run to individual CPU interrupt inputs. This provides ultimate flexibility and control with respect to interrupt priorities and secure/non-secure targeting.

The following macros in ll.h suggest a strategy for interrupt priorities:

```
#define IRQ_PRI_RT      0       // Real time - highest priority
#define IRQ_PRI_HIGH    1
#define IRQ_PRI_NORMAL  2
#define IRQ_PRI_UI      3       // User interface - lowest priority
```

7. Bluetooth LE Stack

Section ATM Wireless Stack above introduced the concept of a distinct partition providing the wireless stack code (aka: ATM Wireless Stack, aka: ATMWSTK=PD100) for the upgradable scheme. The ATMWSTK=PD100 option can also be used with the

CONFIDENTIAL

non-upgradable scheme, which provides more headroom for application debugging while still using the production wireless stack. When ATMWSTK is not specified, and for ultimate developer flexibility with the non-upgradable scheme, the SDK provides debug and non-debug static library versions of the wireless stack.

Details on the wireless stack configurations can be found in the atmwstk_<PD100, PD150,CPD200>.README and ble_52.README files under the SDK platform/atm33/ATM33xx-5/lib directory.

8. Power Management

This section explains the main event loop executed by the pseq driver, with particular focus on the supported power saving modes.

CONFIDENTIAL

8.1 Bluetooth LE Scheduler

The pseq event loop begins with an invocation of the Bluetooth LE software stack's rwip_schedule(). That routine will process all active events and expired timers repeatedly until there is no work remaining.

8.2 Platform Scheduler

Next, rv_plf_schedule/plf_schedule() is invoked to allow user applications and platform software to process items not managed by the Bluetooth LE software stack. This can be a handy place to poll platform devices for status or to insert periodic debug code. See driver/sensor/adt7420/adt7420.c adt7420_schedule() for an example.

8.3 Poke the Watchdog

Then, the hardware watchdog module is poked by writing to its INTCLR register. This keeps it from resetting the system. This is accomplished by the pseq driver calling the watching driver’s watchdog_poke() API.

8.4 Power Management

Now that a pass has been made through the event loop, the system may be idle and ready to transition to a power saving state. While performing the items in this step, CPU

interrupts are globally disabled so that any transition is atomic with respect to the system state.

The state of the Bluetooth LE software stack (including Bluetooth LE applications) is checked to make sure that power management has been enabled (see PARAM_ID_SLEEP_ENABLE section below) and that no active work is ongoing.

Active work can include low level items such as simple events or expired timers, and mid level items such as those described by the prevent_sleep enum or that are managed via rv_prevent_ble_sleep.

The check also computes a sleep duration based upon known future events and any configured limitations:

If NVDS tag `PARAM_ID_EXT_WAKEUP_ENABLE` is unset or set to 0, the maximum sleep duration is:

```
#define MAX_SLEEP_DURATION_PERIODIC_WAKEUP      16384   // 0.5s
```

If NVDS tag `PARAM_ID_EXT_WAKEUP_ENABLE` is set to 1, the maximum sleep duration is:

```
#define MAX_SLEEP_DURATION_EXTERNAL_WAKEUP      327680  // 10s
```

The chosen maximum sleep duration can be overridden by NVDS tag `PARAM_ID_MAX_SLEEP_DUR`.

NVDS tag `PARAM_ID_SLEEP_ADJ` will be subtracted from the sleep duration. This can be used to provide (or remove) leeway between sleep and the next event.

If NVDS tag `PARAM_ID_EXT_WAKEUP_ENABLE` is set to 2, when the computed sleep duration is equal to the max, the computed sleep duration is set to 0, which means forever. The application layer may choose to adjust this to another duration via rv_plf_prevent_retention or rv_plf_prevent_hibernation.

If any active items are found, or the sleep duration is too short, this step is exited and the event loop will repeat. If not, the Bluetooth LE hardware module is placed in the deep sleep state and checks will be made to see if additional layers of power savings can be applied.

CONFIDENTIAL

8.5 PARAM_ID_SLEEP_ENABLE

The NVDS tag `PARAM_ID_SLEEP_ENABLE` is the top level control for power management. It will be read into the sleep_enable global variable during Bluetooth LE initialization. The values are as follows:

```
#define SLEEP_ENABLE_NONE                0
```

Mode NONE means that neither Bluetooth LE nor PSEQ-based power saving is enabled.

When the system is idle, the CPU will wait inside WFI. LED1 will be on during this wait.

```
#define SLEEP_ENABLE_DEEP                1
```

Mode DEEP means that the Bluetooth LE can enter deep sleep. In other words, shut off its primary 16 MHz clock when idle. A small 'always-on' domain using the 32 kHz clock will wake the Bluetooth LE and correct its counters before the next event.

Once the Bluetooth LE is in deep sleep, LED0 will be turned on and rv_plf_to_deep_sleep (defaulting to plf_to_deep_sleep()) will be invoked. The CPU will wait inside WFI. Upon waking up, rv_plf_from_deep_sleep (defaulting to plf_from_deep_sleep()) will be invoked and LED0 will be turned off.

```
#define SLEEP_ENABLE_RETAIN              2
```

Mode RETAIN builds on top of DEEP. Additional system resources will be idled or shut down, while still retaining enough system state to quickly continue. For example, this can be used between radio Rx & Tx events while still maintaining a Bluetooth LE connection.

`plf_to_deep_sleep()` will check sleep_enable and the computed sleep duration (see Platform sleep duration section below). rv_plf_prevent_retention will be invoked to provide high level applications with a chance to adjust or avoid this mode. If any of these conditions are not met, plf_to_deep_sleep() will return and fall back to SLEEP_ENABLE_DEEP.

Finally, rv_plf_retain_all (defaulting to plf_retain_all()) will be invoked to enter the retain power save mode. This routine will program the wakeup timer & mask, turn on LED3, and initiate the power save mode. The primary 16 MHz clock will be shut down for the entire chip, some stateless modules will be powered off, and RAM will be placed in a state retention mode.

CONFIDENTIAL

The pseq event loop is designed such that the CPU stack does not need to be retained, which saves power for an entire bank of RAM.

Waking up from the retain power save mode, plf_from_deep_sleep() will invoke rv_plf_back_from_retain_all (defaulting to plf_back_from_retain_all()). plf_back_from_retain_all() will turn off LED3, save the wakeup reason into the wake_status global variable, and set the KE_EVENT_WAKE_STATUS event.

```
#define SLEEP_ENABLE_RETAIN_DROP        3
```

Mode RETAIN_DROP builds on top of RETAIN by also lowering the voltage of certain internal power rails. Note that this voltage lowering will not take place when the wakeup receiver (WURX) is enabled.

```
#define SLEEP_ENABLE_HIBERNATE          4
```

Mode HIBERNATE builds on top of RETAIN_DROP, but in a different manner. All of the previous modes will preserve system state, but hibernation will cause most of the system state to be lost. Therefore, it is essential for hibernation to interact with high level application state, such as whether there is an active Bluetooth LE connection or not.

plf_to_deep_sleep() will check sleep_enable and the computed sleep duration (see Platform Sleep Duration section below). rv_plf_prevent_hibernation will be invoked to provide high level applications with a chance to adjust or avoid this mode. If any of these conditions are not met, plf_to_deep_sleep() will fall back to `SLEEP_ENABLE_RETAIN_DROP`.

Finally, rv_plf_hibernate (defaulting to plf_hibernate()) will be invoked to enter the hibernate power save mode. This routine will program the wakeup timer & mask, turn on LED1, and initiate the power save mode. All digital circuits will be shut down except for the 'always-on' domain.

Waking up from the hibernate power save mode is similar to a power-on reset. The reset of digital circuits will automatically turn off LED1. The CPU will boot via the ROM.

```
#define SLEEP_ENABLE_SOC_OFF            5
```

Mode SOC_OFF is a deeper, lower power form of HIBERNATE with limited wakeup capabilities.

plf_hibernate() will check the sleep enable, the computed sleep duration, and the wakeup mask for compatibility. If any of these conditions are not met, it will fall back to SLEEP_ENABLE_HIBERNATE.

CONFIDENTIAL

plf_soc_off() will be invoked to program the wakeup timer on top of the configuration from NVDS tag SYDNEY_TAG_PMU_WKUP. All digital circuits will be shut down including the 'always-on' domain. All pins under pinmux control will revert back to the reset state.

Waking up from the SOC_OFF power save mode is almost exactly the same as a power-on reset.

8.6 Platform Sleep Duration

When the above logic decides to enter HIBERNATE or SOC_OFF, plf_hibernate_adjust is subtracted from the computed Bluetooth LE sleep duration to determine the power save duration.

When the above logic decides to enter RETAIN or RETAIN_DROP, plf_retain_adjust is subtracted from the computed sleep duration to determine the power save duration.

The plf_sleep_min variable represents the minimum duration for entering any power save mode.

8.7 Backplane Frequency Scaling

In the above logic, before the CPU invokes the WFI instruction, pseq_scale_back_bp() will be called to lower the backplane clock operating frequency, which helps to save power. rv_plf_bp_throttle will be queried to see if any peripherals are still active and what minimum frequency is required by those activities.

When the CPU comes out of WFI, the backplane clock is restored to its previous operating frequency.

***Note**: Contact ATmosic AE/FAE for details on setting BP frequency.*

9. Crypto Accelerators

9.1 AES Engine

The SDK provides the atm_aes driver, which provides APIs to utilize the AES hardware block to accelerate compute intensive AES operations. The AES hardware can handle either 128 or 256 bit length keys. Five AES modes are supported: ECB, CBC, CFB, OFB, and CTR. AES_CFB is only supported in 128 bit mode.

CONFIDENTIAL

If the input data is not contiguous, or is not all available at the same time, the AES driver supports multiple update calls. However, the AES hardware can only operate on full blocks of data. In ECB, CBC, and CFB modes, the application must handle padding to the next full block size, if needed. OFB and CTR modes inherently support input data of any length and will be automatically padded by the AES driver as needed. Padded bytes in the final block are discarded by the driver and are not available to the application.

While the AES driver supports input data of any alignment, there may be performance benefits if buffers are word aligned.

Please refer to the crypto_demo example demonstrating AES in ECB and CTR modes operating on data using multiple AES update calls.

9.2 SHA2/HMAC

The SDK provides the atm_sha2 driver, which provides APIs to utilize the hashing accelerator that supports SHA2 (SHA-256) and HMAC-256. For HMAC-256 only 256-bit keys are supported. The driver supports two modes of processing: **DMA** and **programmed-IO**. DMA mode, which is the default mode, leverages the DMA engine to accelerate data transfer to the SHA driver. This is the preferred method. However, there are cases when DMA cannot be used (e.g. when executing in secure mode). For those instances, there is a programmed-IO mode that uses the processor to populate the SHA fifo.

If the data is not contiguous, or the data is not all ready at the same time, the sha driver supports multiple updates to the SHA hash. atm_sha256_update() can be called an unlimited number of times with no requirements on input length for each call.

Please refer to the crypto_demo example demonstrating SHA2/HMAC operating on data using multiple SHA/HMAC update calls.

10. Secure Debug

Secure debug is a feature that can be enabled in MCUBOOT that allows for securing of the single-wire debug port (SWD) in a post-production environment. The ATM33/e family provides a hardware mechanism through OTP settings to disable or secure the debug port. When the port is secured, software can implement an out-of-band challenge (communications over UARTs) to authenticate a user requesting access through the

CONFIDENTIAL

SWD port. Details can be found in the **Using Secure Debug with ATM33/e Application Note**.

Reference Documents

Title	Description	Link
Arm® TrustZone Technology for the Armv8-M Architecture v2.1	TrustZone Architecture Manual	https://developer.arm.com/documentation and search with the document title.
Arm® Cortex®-M33 Devices Generic User Guide Revision r1p0	Cortex M33 Users Guide	https://developer.arm.com/documentation and search with the document title.
ARM®v8-M Architecture Reference Manual ARM DDI 0553A.e	ARMv8-M Core architecture reference	https://developer.arm.com/documentation and search with the document title.

Title	Document Number
Using Secure Debug with ATM33/e Application Note	ATM33_e-ANSD

CONFIDENTIAL

Revision History

Date	Version	Description
March 24, 2023	0.54	Updated for SDK 5.4.0.1 release: added Image Swap, Secure Boot, RRAM-Only Scheme, RRAM and External Flash Scheme. Updated MCUboot, Figure 6 - Upgradeable Partition Scheme Block Diagram, Adjusting the Layout, USER DATA.
December 15, 2022	0.53	Updated Figure 6 - Upgradeable Partition Scheme Block Diagram, ATM Wireless Stack section. Added Crypto Accelerators section.
August 24, 2022	0.52	Updated Benign boot check, Bluetooth LE Stack sections. Added Secure Boot sections.
May 13, 2022	0.51	Format changed on the diagrams.
May 9, 2022	0.50	Initial version created

CONFIDENTIAL

ATMOSIC TECHNOLOGIES – DISCLAIMER

This product document is intended to be a general informational aid and not a substitute for any literature or labeling accompanying your purchase of the Atmosic product. Atmosic reserves the right to amend its product literature at any time without notice and for any reason, including to improve product design or function. While Atmosic strives to make its documents accurate and current, Atmosic makes no warranty or representation that the information contained in this document is completely accurate, and Atmosic hereby disclaims (i) any and all liability for any errors or inaccuracies contained in any document or in any other product literature and any damages or lost profits resulting therefrom; (ii) any and all liability and responsibility for any action you take or fail to take based on the information contained in this document; and (iii) any and all implied warranties which may attach to this document, including warranties of fitness for particular purpose, non-infringement and merchantability. Consequently, you assume all risk in your use of this document, the Atmosic product, and in any action you take or fail to take based upon the information in this document. Any statements in this document in regard to the suitability of an Atmosic product for certain types of applications are based on Atmosic's general knowledge of typical requirements in generic applications and are not binding statements about the suitability of Atmosic products for any particular application. It is your responsibility as the customer to validate that a particular Atmosic product is suitable for use in a particular application. All content in this document is proprietary, copyrighted, and owned or licensed by Atmosic, and any unauthorized use of content or trademarks contained herein is strictly prohibited.

Copyright ©2022-2023 by Atmosic Technologies. All rights reserved. Atmosic logo is a registered trademark of Atmosic Technologies Inc. All other trademarks are the properties of their respective holders.

CONFIDENTIAL

Atmosic™

Atmosic Technologies | 2105 S. Bascom Ave. | Campbell CA, 95008
www.atmosic.com